SYSTEM DESIGN

Saga Pattern Visual Guide

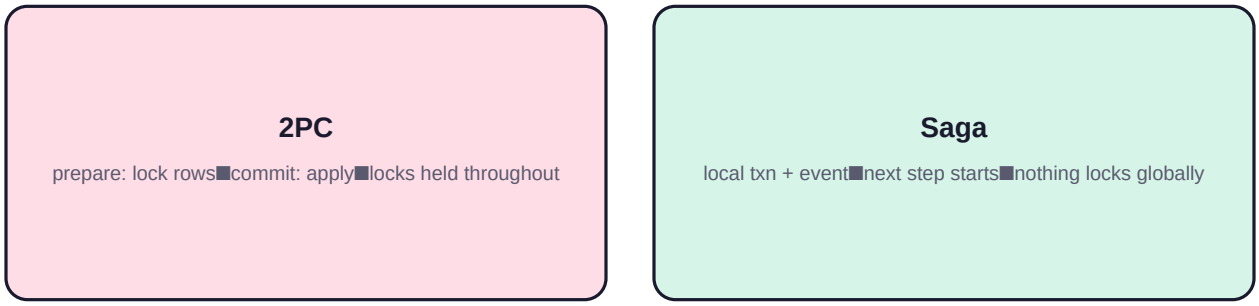
Distributed transactions without 2PC:
sequences, compensation, and failure handling.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Why Not 2PC

Two-phase commit locks every participant until ALL agree -- across services, that blocks.

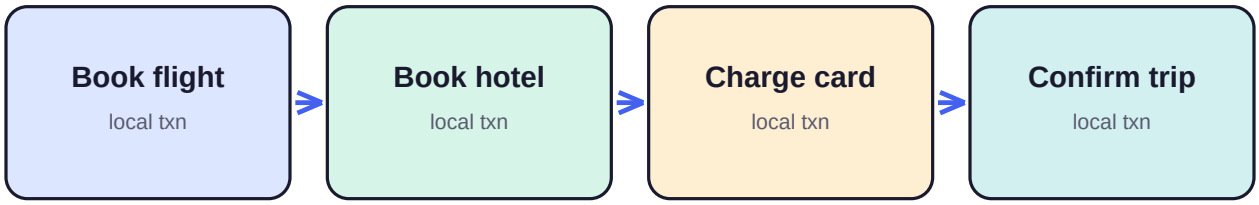


- 2PC across microservices: one slow participant holds everyone's locks; a coordinator crash blocks everything.
- Saga insight: each step commits its LOCAL transaction immediately, then triggers the next step.
- Atomicity becomes 'all steps complete OR all completed steps are undone' -- via compensation.

STEP 2/6

The Saga Sequence

A trip booking: each step commits locally and publishes an event.



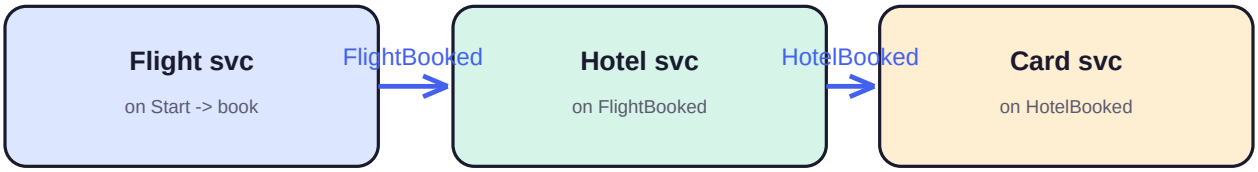
flight OK -> hotel OK -> card OK -> confirm. Each arrow is an event; each box already committed.

There is no isolation between steps: other transactions can SEE the booked flight before the trip confirms.

STEP 3/6

Choreography Saga

Services listen for events and run their step -- no central brain.

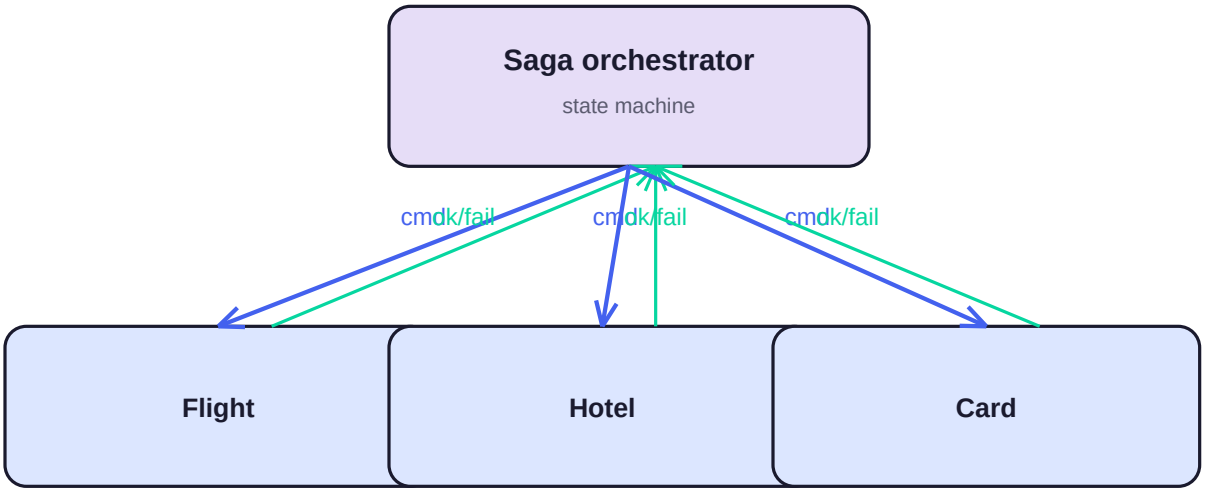


- Pros: no orchestrator to build or scale; services stay autonomous.
- Cons: the saga's logic is scattered -- 'what happens after hotel fails?' lives in five codebases.
- Cyclic risk: A listens to B listens to A -- draw the event graph or it will surprise you.

STEP 4/6

Orchestration Saga

One coordinator runs the script: command, wait, decide.

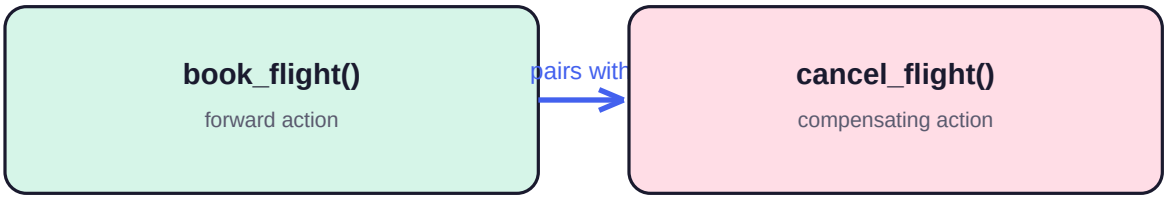


- The flow is explicit code: easy to read, test, and reason about failures.
- The orchestrator holds saga STATE (durable: survive its own crash -- use a workflow engine).
- Temporal / Camunda / AWS Step Functions exist so you do not hand-roll this state machine.

STEP 5/6

Compensating Actions

You cannot roll back a committed local txn -- you run its semantic UNDO.

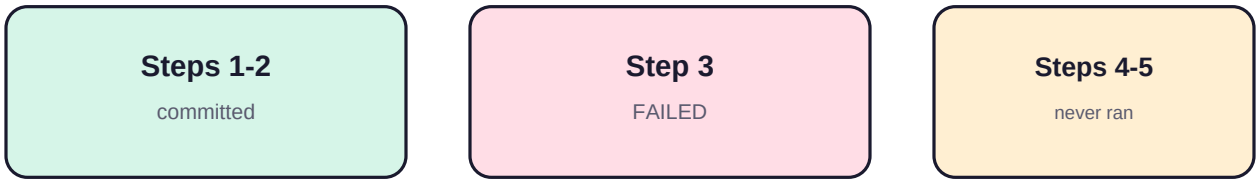


- Every step needs a compensating twin: charge -> refund, reserve -> release, send email -> send correction.
- Compensation runs BACKWARD through completed steps (LIFO): undo card, then hotel, then flight.
- Some actions are un-compensatable (email sent) -- design forward recovery or human steps for those.

STEP 6/6

Failure Handling

Step 3 of 5 fails. Now what?



- Backward recovery: run compensations for steps 1-2 (the classic rollback).
- Forward recovery: retry step 3, or reroute (book a different hotel) and continue.
- Make every action IDEMPOTENT: compensations and retries will execute more than once.
- Durable saga log: record each step's outcome so a crashed orchestrator resumes, never restarts.

STEP ✓I✓

Key Takeaways

Saga Pattern — Remember

- Sagas replace 2PC's global locks with local transactions chained by events or commands.
- Choreography: services react to events (decoupled, scattered logic). Orchestration: a coordinator runs the script.
- Committed steps cannot roll back -- each needs a compensating action that semantically undoes it.
- On failure: compensate backward (undo) or recover forward (retry/reroute); make everything idempotent.
- Durably log saga state -- the orchestrator must survive its own crash mid-saga.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev